

UNIT-VI

Obstacles to Agile Software Development

Management Intransigence

The Failed Project Syndrome

Contractual Difficulties

Familiarity with Agility

Obstacles to Agile Software Development

In this UNIT , we consider the

- issues associated with management intransigence regarding new development processes.
- Address the issue of the “failed agile project” syndrome, that is, what do you do if some- where else in your organization an agile project has been tried and failed.
- Developer resistance is also a major drag factor as well as customer opposition (this later possibility, the biggest stumbling block of all, as you really do need customer involvement in any agile project).
- Next, we consider contractual difficulties arising from the traditional use of requirements as the basis for any develop- ment contracts. Finally, we discuss the lack of familiarity with agile development methods.

Management Intransigence

- The drive towards agile software development often comes from the developers themselves or from developer teams. In such situations, management can represent a significant obstacle to adoption of an agile approach.

This can be for a host of reasons that include:

- Lack of familiarity with Agile Software Development methods.
- Miscomprehension of what eXtreme Programming and Agile Modelling offers (such as believing them to be legalised hacking).
- A feeling of losing control (as agile methods are more dynamic in terms of planning activities than traditional approaches).

- Remoteness from the actual coalface and thus remoteness from development issues.
- The need to feel that they have the whole project planned out in advance.
- Lack of suitability for their own review-and-assessment process. That is, if they are assessed on a more traditional waterfall model, then they may have the production of a detailed project plan as one of their aims and objectives to be completed.
- Belief that adopting eXtreme Programming (and thus doubling developers up) will actually halve productivity. Although current research indicates that the opposite is actually true.
- The parapet syndrome. That is, a manager may not want to risk a different approach to that normally adopted (i.e., putting their head above the parapet), as no one may be censured for doing things the standard way, but they may well be for doing things in a different way and failing!
- The fear of the unknown!

The Failed Project Syndrome

The issues had been with the project.

- No one involved in the project had ever done an agile development before.
- Half the developers were actually industrial engineering students taking time out between their second and third years, and had limited commercial software development experience anyway.
- The belief appeared to be that you should never comment code (it was self- documenting).
- They never wrote any documentation (believing it not to be the agile way).
- They never did any design (as they considered design to be the antithesis of agility).
- Refactoring appeared to have been an area they considered a waste of time!

Contractual Difficulties

- A very significant issue relating to agile software development methods is that of contractual difficulties. By that, I mean that contracts traditionally have been based around a waterfall software development approach.
- Thus, the buyer of the software may state exactly what is required, and if all the specified functions are not provided, then the contract is not met. This situation is often exacerbated by the presence of fixed price projects. That is, a supplier must state exactly how much they will charge to provide the required functionality. All of this of course takes place in advance of the software development and may not even involve the actual developers.

Emphasising the following to be useful:

- ***The level of control/influence they can exert.*** That is, the extent to which the clients can influence what is done when, can sometimes be a major surprise. Software buyers often used to hand over the requirements, and then feel that they lose control of the project, and that they are left out of any project decision made internally. In an agile approach, the client and his representatives should be on board all the time (indeed XP and Agile Modelling require their direct inclusion in the team).
- ***The feedback they will receive.*** Returning to the issue of the involvement of the client in the project, in many cases, clients have received very little feedback on the progress of the project. Often what feedback they have received may later prove to at best have been optimistic and at worst a down right lie. As the clients find themselves in the midst of the project, they will naturally receive regular feedback. They will also be able to judge the accuracy of the information they receive as and when releases are made (at frequent intervals).

- ***The adaptability of the project to changing requirements.*** One regular issue for bespoke software buyers is the problem of changing requirements. If the requirements document is the basis of the contractual agreement between supplier and customer and subsequently if a requirement changes, then the contract changes. In some cases, this can be a very large hurdle to cross and will require lawyers and protracted negotiations. However, as requirements may be drawn up well in advance of the software development, they may well have changed/may well change in the future. By adopting an agile approach, as requirements change, these changes can be naturally and simply passed through to the project team.
- ***The ability to prioritise features.*** A (possibly) surprising advantage of the agile approach to customers is that a low priority feature in a software system can be included but only implemented if time allows or its priority rises.

Familiarity with Agility

One major obstacle to adopting an agile software development is the lack of knowledge of how to start and run such a project. We discussed ways to introduce this approach into an organization. However, until you have been involved in one or more agile software developments, a great many questions may stop you in your tracks. For example, “how do we estimate the cost of the software to the clients at the start of a project?” “how do we decide how many iterations will be there?” “how do we know what will be in each iteration?” and “how do we decide how long an iteration should be?” There are no right or wrong answers to these questions – although there are answers that may be more right or more wrong than others.

How do you estimate the cost of the software to the clients?

The issue here may appear to be a question of how can we determine how much this software will cost for us to develop, given that we don't yet know exactly what we will do, when and how long (particularly later) functionality will take to implement. The real issue here is that the focus of the question is wrong. Customers need a budget to work within and so do you. Thus, if the client has a budget of £60,000 and requires the final release in 6 months time, then you already know how much effort you can afford to take on the project. That may or may not be enough to implement all the current requirements (which may be subject to change anyway). However, in general, the majority of software systems do not require all the possible functions that could be put into them, but they do need to remain within their budgets

How do we decide how many iterations there will be?

This may appear as an obstacle because the question being implied here is “how many iterations will it take to implement *all* the requirements?” Whereas in fact, the issue should be “How many iterations should there be, to enable the clients to receive an appropriate number of releases, and for them to provide feedback to the development team, and to consider the current open questions?” This question implicitly acknowledges that some features may remain un-implemented as time may not allow for them. It also acknowledges that some as yet unknown user-feedback will impact upon that set of features. Thus, the size of each iteration relates more to the overall length of the project, the size and complexity of the system being implemented and the business processes of the client that will allow interim releases to be made

How do we know what will be in each iteration?

- Again this question arises from the more traditional waterfall mind set – it is really saying, “I want to know exactly what will be done in all iterations of the project.” Whereas, agile methods essentially say that we will *roughly* decide what features, functions, use cases or user stories (depending on your preferred terminology or approach) will be planned for each iteration, but that only the current iteration will be planned in detail. Then, at the end of the current iteration, we will review the overall plan for the iterations and features, and that the next iteration will then be planned in detail. Thus, we only plan for the next few weeks or months (depending upon the size of an iteration) in detail.

How do we decide how long an iteration should be?

This goes back to an earlier question, but is worth considering here again. Each iteration should be large enough to contribute something of value to the end-user, but small enough to allow for rapid and useful feedback from those users. Larger, more complex systems will thus naturally require larger iterations, whereas smaller, less complex systems will be able to achieve “something of value to the end-user” in smaller steps. Although, it is also worth noting that as a project matures, later iterations may be able to achieve more in shorter time periods, as there are larger “chunks” that can be exploited.

Questions on VI UNIT

1. *What are the Obstacles to Agile Software Development?*
2. *How does the management can represent a significant obstacle to adoption of an agile approach?*
3. *What are the issues faced from the Failed Project Syndrome.*
4. *What are significant issue relating to agile software development methods is that of contractual difficulties and how they are the Emphasising to be useful?*
5. *How do you estimate the cost of the software to the clients?*
6. *How do we decide how much iteration there will be?*
7. *How do we know what will be in each iteration?*
8. *How do we decide how long an iteration should be?*

5. How do you estimate the cost of the software to the clients?

6. How do we decide how much iteration there will be?

7. How do we know what will be in each iteration?

8. How do we decide how long an iteration should be?